Getting Started with Cortex Code — A Practical Guide

Cortex Code (CoCo) is Snowflake's AI coding agent. It runs in your terminal, reads and writes files, runs tools (Bash, git, SQL, dbt), and connects directly to your Snowflake accounts. Think of it as a fast, knowledgeable pair-programming partner — great at execution, but you still own the design, the review, and the judgment.

This guide covers everything you need to go from first launch to confident daily use.

TABLE OF CONTENTS

1. Launching CoCo
2. The Core Loop
3. How to Be Specific (The Art of Good Prompts)
4. CLI Commands and Keyboard Shortcuts
5. Skills: What They Are and When to Use Them
6. Context and Session Management
7. AGENTS.md: Your Project Briefing Document
8. Sub-Agents
9. What to Be Careful Of
10. Common Failure Modes and Fixes
11. Example Conversations
12. Quick Reference Card

---

1. LAUNCHING COCO

Basic Launch

```
cortex
```

Launch Against a Specific Snowflake Connection

```
cortex -c MY_CONNECTION
```

Launch with a Profile (Restricted Permissions)

```
cortex -profile read-only -c MY_CONNECTION
```

Resume Your Last Session

```
cortex --resume
```

When CoCo starts, it shows a banner confirming your Agent Connection (for AI inference) and SQL Connection (for executing queries). If both show the correct accounts, you're set.

---

2. THE CORE LOOP

Every task follows this pattern. Master this before touching anything else.

```
1. State what you want in one sentence
2. /plan — CoCo proposes steps without changing anything
3. Review the plan. Say what's off-limits.
4. Execute one phase at a time
5. Review every diff. Read every SQL statement.
6. Verify: run tests, check results
7. Commit
```

This is the whole method. Skills, sub-agents, and custom workflows all build on this foundation.

Example: The Core Loop in Action

You: "I need a staging table in DEV_DB.STAGING that ingests daily CSV files from our S3 stage @raw_data. Use ANALYST_ROLE and ANALYST_WH. Do not touch any production schemas."

CoCo: *proposes a plan with steps: create table, create pipe, test with sample data*

You: "Good plan. Execute step 1."

CoCo: *creates the staging table, shows the DDL*

You: *reviews the SQL, confirms it targets DEV_DB.STAGING* "Looks good. Execute step 2."

This is how every session should feel: deliberate, incremental, reviewed.

3. HOW TO BE SPECIFIC (THE ART OF GOOD PROMPTS)

The single biggest factor in CoCo's output quality is the specificity of your input. Before every request, think about four questions:

Question	What to include
What do I want?	State it in one sentence.
What are the boundaries?	What should NOT be changed? Which database/schema/role?
What do I already know?	Table names, column names, date ranges, filters.
How will I know it worked?	What should the output look like?

Bad vs. Good Prompts

Vague (bad):

> "Show me the data"

Specific (good):

> "Show me the top 10 customers by revenue in SALES_DB.PUBLIC.ORDERS for Q4 2025. Use ANALYST_ROLE and ANALYST_WH."

Vague (bad):

> "Write a query"

Specific (good):

> "Write a query against ANALYTICS.PUBLIC.REVENUE that shows monthly trends. The date column is ORDER_DATE and the amount column is TOTAL_AMOUNT. Include a running total."

Vague (bad):

> "Build me a dashboard"

Specific (good):

> "Create a Streamlit dashboard that shows monthly revenue trend (line chart), top 10 products (bar chart), and regional breakdown (pie chart). Use ANALYTICS_DB.REPORTING schema. Read-only access. Use ANALYST_ROLE."

Vague (bad):

> "Fix this error"

Specific (good):

> "This query fails with 'ambiguous column name AMOUNT'. The query joins ORDERS and RETURNS tables, both of which have an AMOUNT column. Qualify the column references so it uses ORDERS.AMOUNT for the total."

Power-User Prompt Patterns

Ask CoCo to ask you questions first:

> "Build a data pipeline that loads customer events from S3 into Snowflake. Ask me clarifying questions before you start."

This consistently produces better results. CoCo will surface ambiguities you didn't think of.

Tell CoCo what NOT to do:

> "Use ANALYST_ROLE and ANALYST_WH. Do not modify any tables — read only. Do not create any new objects."

Ask it to explain as it goes:

> "Write this query and explain what each part does in plain English."

Give it the full context:

> "The table EVENTS has columns: EVENT_ID (INT), USER_ID (INT), EVENT_TYPE (VARCHAR), EVENT_TS (TIMESTAMP_NTZ), PAYLOAD (VARIANT). Write a query that extracts the 'amount' field from PAYLOAD for EVENT_TYPE = 'purchase' and aggregates by month."

Plan in one session, review in another:

> Session 1: **`/plan`** for the full architecture, save the plan to a file
> Session 2: "Read notes/plan.md and start executing phase 1."

The fresh context in session 2 gives CoCo a sharper perspective and avoids the bloat of a long planning session.

---

4. CLI COMMANDS AND KEYBOARD SHORTCUTS

Built-in Commands

Command	What it does
`/plan`	CoCo proposes steps without changing files. Use before any multi-step work.
`/model`	Switch the underlying AI model mid-session.
`/context`	Show context window usage — how full is CoCo's working memory.
`/compact`	Compress conversation history to free context space. Keeps a summary.
`/clear`	Start a completely fresh session.
`/rewind`	Rewind to an earlier checkpoint.
`/help`	Get help and see available commands.

Keyboard Shortcuts

Action	Shortcut
Interrupt CoCo mid-action	`Esc`
Rewind to edit an earlier prompt	`Esc + Esc`
Accept changes	`Enter`
Multi-line input	`Shift + Enter`

Custom Slash Commands (After Install)

Command	What it does
`/handoff`	Saves a structured session summary before you reset
`/pre-session-check`	Quick health check of connection, AGENTS.md, and context

Command	What it does
`/plan-review`	Structured planning workflow with clarifying questions
`/context-health`	Checks context window usage and drift signals
`/quick-diagnose`	Structured debugging workflow

Where custom commands live:
- Global (all projects): `~/.snowflake/cortex/commands/*.md`
- Project-level: `.cortex/commands/*.md`

---

5. SKILLS: WHAT THEY ARE AND WHEN TO USE THEM

What Is a Skill?

A **skill** is a packaged workflow — a saved expert playbook that CoCo follows when it recognizes a matching request. Think of it as a recipe: steps, decision points, guardrails, and expected output, all bundled into a `SKILL.md` file.

CoCo ships with built-in skills for common Snowflake tasks. You can also install or create your own.

How to Invoke a Skill

You don't need to call skills explicitly. CoCo automatically matches your request to the right skill based on natural language. Just describe what you want:

> "Create a semantic view for my sales data"

CoCo recognizes this matches the `create_semantic_view` skill and activates it.

> "Help me optimize my semantic view"

CoCo matches this to `improve_semantic_view`.

> "Deploy my Streamlit app"

CoCo matches this to the `developing-with-streamlit` skill.

Examples of Built-in Skills

Skill	What it does	Trigger phrases
semantic-view-optimization	Debug, fix, optimize semantic views	"fix my semantic view", "debug semantic view", "optimize semantic view"
create_semantic_view	Create a new semantic view YAML	"create a semantic view", "build semantic model"
developing-with-streamlit	Create, edit, debug Streamlit apps	"build a streamlit app", "fix my dashboard", "create a dashboard"
dynamic-tables	Create, monitor, troubleshoot Dynamic Tables	"create a dynamic table", "debug DT", "DT pipeline"
dbt-projects-on-snowflake	Build, run, test dbt projects	"run dbt", "build dbt model", "dbt test"
machine-learning	Train, deploy, register ML models	"train a model", "deploy model", "model registry"
agent-optimization	Build, debug Cortex Agents	"create an agent", "debug my agent"
cost-management	Analyze Snowflake costs and credits	"show my costs", "credit consumption", "warehouse costs"
data-governance	Audit access, permissions, roles	"who has access", "audit grants", "role hierarchy"
search-optimization	Build Cortex Search services	"create a search service", "search pipeline"
sensitive-data-classification	Detect PII, classify sensitive data	"detect PII", "classify table", "sensitive data"

When to Use a Skill vs. Just Prompting

Situation	Approach
One-off exploration, simple query	Just prompt CoCo directly
Repeatable, multi-step workflow with guardrails	Use a skill
You're doing something for the first time	Prompt first, consider a skill later
You've done this 3+ times and want consistency	Build or use a skill

Invoking a Skill Explicitly (If Needed)

If CoCo doesn't auto-detect the skill, you can nudge it:

> "Use the developing-with-streamlit skill to help me build a dashboard"

Or more directly:

> "Invoke the dbt-projects-on-snowflake skill"

Where Skills Live

Level	Location
Built-in (bundled)	Shipped with CoCo — always available
Global (custom)	`~/.snowflake/cortex/skills/<name>/SKILL.md`
Project-level (custom)	`.cortex/skills/<name>/SKILL.md`

---

6. CONTEXT AND SESSION MANAGEMENT

This is the most underappreciated aspect of working with any AI coding agent. Understanding it will save you hours of frustration.

What Is the Context Window?

CoCo has ~200K tokens (~150,000 words) of working memory per session. Everything goes into it: your messages, file contents, tool outputs, conversation history, and AGENTS.md instructions.

What Happens When It Fills Up?

As the window fills, CoCo automatically **compacts** older messages — summarizing them to free space. This is lossy. Decisions get compressed. Nuance disappears. Output quality drops gradually, then sharply.

Signs Your Session Is Degrading

- CoCo contradicts something it said earlier
- It forgets instructions you gave 10 minutes ago
- Language becomes unusually dramatic or theatrical
- It agrees with everything without pushback
- It "remembers" things from prior sessions (it can't — it's stateless)

- Output feels repetitive or circular

What to Do About It

1. **Check context usage:** Type `/context` to see how full the window is.
2. **Compact if needed:** Type `/compact` to compress history but keep a summary.
3. **Reset proactively:** If quality is dropping, use `/handoff` to save a structured summary, then `/clear` and start fresh.

The Reset Pattern

```
1. "Save a summary of what we've done to notes/session-handoff.md"
   (or use /handoff if installed)
2. /clear
3. "Read notes/session-handoff.md and continue where we left off"
```

This is normal workflow hygiene — like saving your work and reopening a clean document. It is not a sign that something went wrong.

Pro Tips for Context Hygiene

- **Don't paste huge logs.** Summarize first: "Here are the 3 key errors and the last 20 lines."
- **Use sub-agents for broad exploration.** They work in their own context and return only results.
- **Front-load important information.** The first ~40% of context gets the strongest attention from the model.
- **Keep AGENTS.md under 300 lines.** Link to reference docs instead of inlining everything.

---

7. AGENTS.MD: YOUR PROJECT BRIEFING DOCUMENT

Every project should have an `AGENTS.md` file at the root. CoCo reads it automatically at the start of every session. This is the single highest-leverage thing you can do to improve CoCo's output.

What to Include

Section	Example
Project summary (2-3 sentences)	"This is a data pipeline that ingests customer events from S3, transforms them in Snowflake, and serves a Streamlit dashboard for the analytics team."

Section	Example
Tech stack and folder layout	"dbt models in /models, Streamlit app in /app, SQL scripts in /sql"
Snowflake connection	"Account: MYORG-MYACCOUNT, Role: ANALYST_ROLE, Warehouse: ANALYST_WH, Database: ANALYTICS_DB"
How to test and build	"Run `dbt test` for data quality. Run `streamlit run app/main.py` for the dashboard."
Key rules	"Never run against production. Always use CREATE OR REPLACE. Never modify RBAC roles."

What to Leave Out

- The entire codebase pasted inline (wastes context)
- Negative-only rules without alternatives ("Never use flag X" — instead: "Use flag Y instead of flag X")
- Everything you know (focus on what CoCo tends to get wrong)

Example AGENTS.md

Here's a complete, realistic example you can adapt:

```
# Customer Analytics Pipeline

This project ingests customer event data FROM S3, transforms it WITH dbt IN Snowflake,
AND serves a Streamlit dashboard FOR the analytics team.

## Snowflake Connection

- **ACCOUNT:** MYORG-ANALYTICS_PROD
- **ROLE:** ANALYTICS_DEV_ROLE
- **WAREHOUSE:** ANALYTICS_WH (Medium)
- **DATABASE:** ANALYTICS_DEV_DB
- **SCHEMA:** STAGING, MARTS

## Tech Stack

- **Pipeline:** dbt (dbt-snowflake adapter)
- **Dashboard:** Streamlit IN Snowflake
- **LANGUAGE:** SQL + Python 3.11

## Folder Layout

project-root/
├── AGENTS.md
├── models/
│   ├── staging/         # Cleaned, typed source tables
│   └── marts/           # Business-logic models (joins, aggregations)
├── app/
│   └── dashboard.py     # Streamlit dashboard
├── tests/               # dbt tests + custom Python tests
├── macros/              # dbt macros
├── agent_docs/          # Reference docs CoCo can read ON demand
│   ├── architecture.md
│   └── testing.md
└── scripts/
    └── deploy.sh

## How TO Build AND Test

# Run dbt models
dbt run --select staging+

# Run dbt tests
dbt test

# Run the Streamlit app locally
streamlit run app/dashboard.py

## KEY Rules

### Always
- Use fully qualified names: ANALYTICS_DEV_DB.STAGING.TABLE_NAME
```

```
- Use CREATE OR REPLACE instead of DROP + CREATE
- Write a dbt test FOR every new model
- Commit at every meaningful checkpoint

### Never
- EXECUTE anything against ANALYTICS_PROD_DB — dev only
- GRANT OR REVOKE roles — RBAC IS managed BY the platform team
- Add Python dependencies without asking first
- Skip diff review ON SQL changes

### Prefer
- Incremental models over FULL refreshes FOR large tables
- CTEs over nested subqueries
- snake_case FOR ALL OBJECT names
- Parameterized filters over hardcoded DATE ranges

## Reference Docs

These are IN agent_docs/ — read them WHEN relevant:
- agent_docs/architecture.md — Data flow FROM S3 → staging → marts → dashboard
- agent_docs/testing.md — How TO write AND run dbt tests, what coverage we expect

## Domain Notes

- &quot;Customer segment&quot; uses the SEGMENT column IN DIM_CUSTOMERS, NOT a derived valu
e
- Revenue means GROSS_REVENUE minus REFUNDS (net revenue), unless stated otherwise
- Fiscal quarters start IN February (Q1 = Feb-Apr), NOT January

## Active Decisions

- 2025-12-01: Chose incremental strategy FOR FCT_EVENTS (50M+ rows/day)
- 2026-01-15: Dashboard uses Snowflake's native Streamlit, NOT EXTERNAL hosting
- 2026-02-20: ALL new models use the TRANSIENT TABLE TYPE TO reduce TIME Travel costs
```

Reference Doc Pattern

Create a small folder of focused files:

```
agent_docs/
  architecture.md
  running_tests.md
  snowflake_rbac_rules.md
```

Mention these in AGENTS.md so CoCo knows they exist. It will read them on demand — like a librarian fetching a specific book instead of dumping the entire library on your desk.

File Locations

Level	Location
Global (all projects)	`~/.snowflake/cortex/agents.md`
Project root	`AGENTS.md` (also supports `CLAUDE.md`, `RULES.md`)
Subdirectory	`frontend/AGENTS.md` (for monorepo scoping)

---

8. SUB-AGENTS

A **sub-agent** is a separate CoCo instance that your main session launches for a specific job. It works in its own context window and reports back a summary.

When to Use Sub-Agents

- **Parallel reviews:** Launch reviewers for code, security, and cost simultaneously
- **Broad exploration:** Scan a large codebase without filling your main session's context
- **Repetitive tasks:** Log scanning, style reviews, dry runs

How to Use Them

> "Launch the security-reviewer agent to review the SQL in /sql/deploy.sql"

> "Launch 3 sub-agents: code-reviewer, security-reviewer, cost-reviewer. Have them review the current changes."

How to Structure a Sub-Agent

Give each one a clear, narrow contract:

Define this	Example
Name	`security-reviewer`
One job	"Review this SQL for security issues only"
Inputs	Specific files or queries to examine
Expected output	A markdown summary with findings

Where Sub-Agents Live

Level	Location
Global	`~/.snowflake/cortex/agents/<name>.md`
Project	`.cortex/agents/<name>.md`

Important: Sub-Agents Are a Power-User Feature

Master the core loop first. Sub-agents without structure produce confused results and waste tokens. If you're new to CoCo, skip this section and come back in a few weeks.

---

9. WHAT TO BE CAREFUL OF

Always Review Before Accepting

CoCo will show you what it wants to do. **Read it.** If it's SQL, verify:
- The target database and schema
- The role and warehouse
- Whether it's DDL (creates/alters objects) or DML (inserts/updates/deletes data)

If it's code, make sure it looks reasonable. When in doubt: "Explain what this will do in plain English."

CoCo Is Stateless

Each session starts fresh. CoCo does NOT remember yesterday's conversation. Your AGENTS.md reloads automatically, but everything else needs to be explicitly provided (handoff files, reference docs, etc.).

Same Prompt, Different Output

Ask the same thing twice, get slightly different (but correct) output. This is how ALL LLM-based agents work — Cursor, Copilot, Codex, CoCo. You get consistent correctness, not identical output.

Sessions Degrade Over Time

As context fills, quality drops. Reset proactively. Don't wait for things to break. See Context and Session Management.

Never Run Destructive SQL Without Reading It

Ask CoCo: "What will this change?" Review the answer before executing. This applies double in shared or production-adjacent environments.

Use a Dev/Stage Role

Never give CoCo direct production access. Use a locked-down Snowflake role scoped to dev/stage environments. RBAC is the hard boundary — CoCo can only do what the role allows.

Be Cautious with `CREATE OR REPLACE`

`CREATE OR REPLACE` is generally safer than `DROP` + `CREATE`, but it still replaces objects. Make sure you're targeting the right schema.

---

10. COMMON FAILURE MODES AND FIXES

What you do	What goes wrong	Fix
"Build me an entire pipeline" in one prompt	Sprawling, inconsistent output	Break it into phases. Use `/plan`. One phase at a time.
Accept every change without reading it	Unreviewed SQL hits shared environments	Review every diff. Every time. No exceptions.
Never reset sessions	Quality degrades. CoCo contradicts itself.	`/handoff` + `/clear` when things feel off.
Expect CoCo to "know" your codebase	It guesses wrong about architecture	Write an AGENTS.md. Provide file paths explicitly.
Paste full error logs	Context fills with noise	Summarize: 3-5 key errors + last 20 lines.
Try sub-agents before mastering basics	Frustration, wasted tokens	Master the core loop first. Sub-agents are Level 5+.
Mix off-topic chat with coding	Persona drift — CoCo acts strangely	Keep it focused on code and data. Reset if drift appears.

The "AI is Flaky" Reframe

If CoCo (or any AI coding agent) feels unreliable:

1. **Acknowledge:** LLMs are non-deterministic and context-limited. That's the technology.

2. **Reframe:** The inconsistency is a workflow problem, not a tool problem.
3. **Fix:** Teams that plan, scope, review, and reset get reliable results. Teams that freestyle get inconsistent output.

---

11. EXAMPLE CONVERSATIONS

Data Exploration

> "I'm using ANALYST_ROLE on ANALYTICS_WH. Show me all tables in SALES_DB.PUBLIC and describe what each one contains. Include row counts."

Writing a Report Query

> "Using SALES_DB.PUBLIC.ORDERS and SALES_DB.PUBLIC.CUSTOMERS, write a query that shows total revenue by customer segment for the last 12 months. Include a percentage-of-total column and a month-over-month growth rate."

Understanding Existing SQL

> "Here's a query I found in our reports folder. Explain what it does step by step in plain English, and suggest any performance improvements: [paste query]"

Building a Streamlit Dashboard

> "Create a Streamlit dashboard that shows our monthly revenue trend, top 10 products, and regional breakdown. Use ANALYTICS_DB.REPORTING schema. Read-only access. Use ANALYST_ROLE and ANALYST_WH. Ask me clarifying questions before you start."

Debugging a Pipeline

> "My Snowpipe AUTO_INGEST pipe SALES_DB.STAGING.RAW_PIPE stopped loading 2 hours ago. The status shows 'STOPPED_FEATURE_DISABLED'. Help me diagnose and fix it. Don't modify any production objects."

Creating a dbt Model

> "I have raw tables in RAW_DB.PUBLIC: ORDERS, CUSTOMERS, PRODUCTS. Create a dbt project that builds staging models (cleaned, typed) and a mart model (orders joined with customers and products). Use TRANSFORM_ROLE and TRANSFORM_WH."

Invoking a Skill for Semantic Views

> "I need a semantic view for ANALYTICS_DB.REPORTING.MONTHLY_REVENUE. The table has columns: MONTH (DATE), REGION (VARCHAR), PRODUCT_LINE (VARCHAR), REVENUE (NUMBER), UNITS_SOLD (NUMBER). Create a semantic view that supports natural-language queries about revenue trends."

Multi-Phase Project with Handoffs

> **Session 1:** "I need to build an end-to-end data pipeline: ingest from S3, transform with dbt, serve with Streamlit. /plan the full architecture. Don't execute anything yet."
>
> *Reviews plan, saves to notes/pipeline-plan.md*
>
> **Session 2:** "Read notes/pipeline-plan.md. Execute Phase 1: Ingest. Use DEV_DB.RAW schema."
>
> **Session 3:** "Read notes/pipeline-plan.md. Execute Phase 2: Transform. Here's the handoff from Phase 1: [paste or point to handoff file]"

Using Sub-Agents for Review

> "I've finished the dbt models and Streamlit app. Launch 3 sub-agents: code-reviewer to check correctness and test coverage, security-reviewer to check for SQL injection and credential exposure, and cost-reviewer to check warehouse sizing and query efficiency. Have them review everything in /models and /app."

Cost Investigation

> "Show me the top 10 most expensive queries in the last 7 days across all warehouses. Include the query text, warehouse, credits consumed, and user who ran it."

---

12. QUICK REFERENCE CARD

Essential Commands

Action	Command
Launch CoCo	`cortex`
Launch with connection	`cortex -c CONNECTION_NAME`
Launch with profile	`cortex -profile NAME -c CONNECTION`
Resume last session	`cortex --resume`
Plan before acting	`/plan`
Check context usage	`/context`
Compress history	`/compact`
Clear session	`/clear`
Switch model	`/model`
Interrupt mid-action	`Esc`
Rewind to edit prompt	`Esc + Esc`

The 4-Question Checklist (Before Every Request)

- [ ] **What do I want?** (one sentence)
- [ ] **What are the boundaries?** (role, schema, what NOT to change)
- [ ] **What do I know?** (table names, columns, constraints)
- [ ] **How will I verify?** (expected output, tests to run)

Key File Locations

What	Global	Project
Instructions	`~/.snowflake/cortex/agents.md`	`AGENTS.md`
Commands	`~/.snowflake/cortex/commands/*.md`	`.cortex/commands/*.md`
Skills	`~/.snowflake/cortex/skills/<name>/SKILL.md`	`.cortex/skills/<name>/SKILL.md`
Sub-agents	`~/.snowflake/cortex/agents/<name>.md`	`.cortex/agents/<name>.md`
Settings	`~/.snowflake/cortex/settings.json`	—

What	Global	Project
Hooks	`~/.snowflake/cortex/hooks.json`	—

Safety Rules

1. Always use `/plan` for multi-step work
2. Review every diff and every SQL statement
3. Use a dev/stage role — never give CoCo production access
4. Reset sessions when quality drops — this is normal
5. Don't paste huge logs — summarize first
6. Use `CREATE OR REPLACE`, not `DROP` + `CREATE`
7. Verify target database, schema, role, and warehouse before executing

Progression Path

Level	What to Learn	When
1. Core loop	Goal, /plan, execute, review, verify, commit	Day 1
2. Context setup	Write AGENTS.md, create reference docs	Week 1
3. Session hygiene	Handoff summaries, proactive resets, /compact	Week 1-2
4. Built-in skills	Use Snowflake-provided skills (semantic views, dbt, etc.)	Week 2-3
5. Sub-agents	Parallel reviews, isolated exploration	Month 2+
6. Custom skills	Build your own SKILL.md workflows	Quarter 2+